Day 26: Ensemble Methods — Voting and Stacking

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 4, Day 6

1. Why Combine Multiple Trained Models

> **Definition — Ensembling:** Combining predictions from multiple already-trained models into a single final prediction, on the premise that different models trained on the same data tend to make mistakes on different rows. If those errors aren't perfectly correlated, combining the models' outputs can correct for one model's blind spot with another model's strength.

Real-world example: A hospital diagnostic system combines a decision-tree-based model, a boosting model, and a logistic regression model — each catches slightly different edge cases in patient data — so the combined prediction is more reliable than trusting any single model alone.

- `Base learners` — the individual trained models being combined (here: random forest, sklearn gradient boosting, XGBoost from Day 25).
- Ensembling only helps when base models disagree on some rows — if all three models make the exact same mistakes, combining them changes nothing.

2. Voting Classifier — Hard Voting

> **Definition — Hard voting:** `VotingClassifier(voting="hard")` has each base model cast a single vote — the class label it predicts (0 or 1). The ensemble's final prediction is whichever label gets the majority of votes. With three base models, any 2-1 split decides the outcome.

Real-world example: Three independent doctors each give a yes/no diagnosis without seeing each other's confidence levels; the patient is treated based on whichever answer at least two of the three doctors agree on.

Implementation

```
hard_voting = VotingClassifier(
    estimators=[
        ("forest", forest_pipeline),
        ("gb", gb_pipeline),
        ("xgb", xgb_pipeline),
    ],
    voting="hard",
)
hard_voting.fit(X_train, y_train)
preds = hard_voting.predict(X_test)
```

Line-by-line explanation

- `estimators=[(name, pipeline), ...]` — a list of named (label, fitted-or-fittable pipeline) pairs; each name becomes a key for referencing that estimator later.
- `voting="hard"` — tells the classifier to combine predicted class labels by majority, not probabilities.

• `.fit(X_train, y_train)` — fits every base estimator internally; no separate fitting step is needed for each pipeline beforehand.

• Hard voting requires no probability outputs at all, so it works even with base estimators that don't implement predict_proba.

3. Voting Classifier — Soft Voting

Definition — Soft voting: `VotingClassifier(voting="soft")` averages the predicted class probabilities from every base model, then thresholds the averaged probability at 0.5 to produce the final label. This uses more information than hard voting — a model that's 90% confident contributes more signal than one that's barely over 50%.

Real-world example: The same three doctors now each report a confidence percentage alongside their diagnosis; averaging those percentages before deciding gives more weight to a doctor who is very sure than to one who is only slightly leaning toward a diagnosis.

Implementation

```
soft_voting = VotingClassifier(
    estimators=[
        ("forest", forest_pipeline),
        ("gb", gb_pipeline),
        ("xgb", xgb_pipeline),
    ],
    voting="soft",
)
soft_voting.fit(X_train, y_train)
preds = soft_voting.predict(X_test)
```

Line-by-line explanation

• `voting="soft"` — the only change from hard voting; internally this calls `predict_proba()` on every base estimator instead of `predict()`.

• Every base estimator must implement predict_proba for soft voting to work — random forest, gradient boosting, and XGBoost all do.

• In this lesson's verified run, hard and soft voting produced identical test accuracy (0.750) — soft voting's advantage shows up more when base models' confidence levels genuinely differ on close calls.

4. Stacking Classifier — a Meta-Learner on Top

Definition — Stacking: `StackingClassifier` trains a second model — the `final_estimator`, or meta-learner — on the base models' own predictions as its input features, rather than combining them with a fixed rule like majority vote or averaging. The meta-learner can learn which base model to trust more, and under what conditions, instead of treating every base model equally.

Real-world example: A hiring committee doesn't just average three interviewers' scores — a senior manager (the meta-learner) reviews all three scores together and learns, over many hiring decisions, that one interviewer's high scores are more predictive of success than another's, weighting them accordingly.

Implementation

```
stacking = StackingClassifier(
    estimators=[
        ("forest", forest_pipeline),
        ("gb", gb_pipeline),
        ("xgb", xgb_pipeline),
    ],
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5,
)
stacking.fit(X_train, y_train)
preds = stacking.predict(X_test)
```

Line-by-line explanation

- `final_estimator=LogisticRegression(max_iter=1000)` — the meta-learner; it receives each base model's output as its own input features, not the original raw columns.
- `cv=5` — controls how base-model predictions are generated for training the meta-learner (explained in Topic 5).
- Any classifier can serve as the meta-learner — logistic regression is a common, simple choice since the base models have already done the heavy nonlinear feature work.

5. Why Stacking Uses Cross-Validation Internally

Definition — Out-of-fold predictions: If the meta-learner were trained on base-model predictions made on the same rows those base models were fit on, the predictions would be unrealistically accurate — the base models have already seen the correct answers for those rows. `cv=5` makes StackingClassifier split the training data into 5 folds, generate each base model's predictions on the fold it wasn't trained on, and train the meta-learner only on those honest, out-of-fold predictions.

Real-world example: This mirrors why a teacher grades a student's exam using questions the student hasn't already seen the answer key for — evaluating a model (or a student) on data it was trained on produces an inflated, misleading measure of how well it actually generalizes.

Implementation

```
StackingClassifier(
    estimators=[...],
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5,          # 5-fold out-of-fold prediction generation
)
```

Line-by-line explanation

• `cv=5` — internally equivalent to running `cross_val_predict` for each base estimator across 5 folds to produce its out-of-fold predictions.

• Each base model is then refit on the full training set for use at prediction time — the cross-validation is only used to generate honest training data for the meta-learner, not to select the final base models.

• This is the same leakage-prevention principle from Day 24's SMOTE-inside-a-pipeline lesson: any step that could 'see' information it shouldn't must be fit only on the appropriate training fold.

6. Comparing Base Models, Voting, and Stacking Fairly

Definition — Controlled comparison: Evaluating all six approaches — three base models, hard voting, soft voting, and stacking — on the identical train/test split and identical preprocessing, so the comparison isolates the combination strategy itself as the only variable.

Real-world example: A team building a customer-churn model benchmarks its base classifiers against voting and stacking variants before committing to the added complexity of an ensemble in production, since ensembles cost more to maintain and serve.

Implementation

```
results = {}
for name, model in [
    ("Random Forest", forest_pipeline),
    ("Sklearn GB", gb_pipeline),
    ("XGBoost", xgb_pipeline),
    ("Hard Voting", hard_voting),
    ("Soft Voting", soft_voting),
    ("Stacking", stacking),
]:
    model.fit(X_train, y_train)
    results[name] = accuracy_score(y_test, model.predict(X_test))
```

Line-by-line explanation

• Every model in the loop is fit on the exact same X_train/y_train and scored on the exact same X_test/y_test.

• Verified in this lesson: stacking and standalone XGBoost tied for the best test accuracy (0.783), ahead of hard/soft voting (0.750) and the plain random forest and sklearn boosting (0.733 each).

• Ensembling rarely produces a dramatic jump when base models already agree most of the time — its value is clearest when base models make genuinely different kinds of errors.

7. Tuning the Stacking Meta-Learner

Definition — Meta-learner tuning: Grid-searching only the meta-learner's hyperparameters (here, logistic regression's `C` regularization strength) via the `final_estimator__` prefix, while leaving

the base learners' Day 25 settings untouched. Jointly tuning every base model's hyperparameters alongside the meta-learner would multiply the search space and compute cost enormously.

Real-world example: A team with limited compute budget tunes only the final decision-combining layer of their ensemble rather than re-running a full hyperparameter search across every base model, trading a small amount of potential accuracy for a much faster iteration cycle.

Implementation

```
param_grid = {"final_estimator__C": [0.1, 1.0, 10.0]}
search = GridSearchCV(stacking, param_grid, cv=5, n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
```

Line-by-line explanation

- `"final_estimator__C"` — the double underscore reaches into the StackingClassifier's final_estimator to set logistic regression's C parameter.
- `C` — inverse regularization strength; smaller values apply stronger regularization to the meta-learner.
- `cv=5` in GridSearchCV is a separate cross-validation loop from the `cv=5` inside StackingClassifier itself — one evaluates hyperparameter choices, the other generates honest training data for the meta-learner.

Interview Preparation — Technical Questions

Q: What's the difference between hard voting and soft voting?

A: Hard voting combines base models' predicted class labels by majority rule. Soft voting averages base models' predicted probabilities and thresholds the average — it uses more information (confidence levels) but requires every base estimator to support predict_proba.

Q: What does a StackingClassifier's final_estimator actually train on?

A: It trains on the base models' predictions (or predicted probabilities) as input features, not on the original raw dataset features. It learns how to best combine the base models' outputs rather than learning new patterns from the raw data directly.

Q: Why does StackingClassifier use cross-validation internally?

A: To generate out-of-fold predictions from each base model for training the meta-learner. Without this, the meta-learner would train on predictions the base models made on rows they were themselves fit on, which are unrealistically accurate and would cause the meta-learner to overestimate the base models' true generalization ability — a form of data leakage.

Q: When would ensembling fail to improve over the best single base model?

A: When the base models are highly correlated — they tend to make the same predictions and the same mistakes on the same rows. Ensembling adds value by combining models that disagree; if all base models already agree almost all the time, combining them changes little.

Q: Why is only the meta-learner tuned via GridSearchCV in this lesson, not the base learners?

A: Jointly tuning every base model's hyperparameters alongside the meta-learner multiplies the search space combinatorially and becomes computationally expensive. Base learners keep their already-tuned Day 25 settings, and only the cheaper, smaller meta-learner search is run.

Q: What kinds of models can serve as a StackingClassifier's final_estimator?

A: Any classifier that implements fit and predict (or predict_proba) — logistic regression is common because the base models have already done the heavy nonlinear feature transformation, so a simple linear combiner is often sufficient.

Q: In this lesson's results, why did stacking and XGBoost alone tie for the best accuracy instead of stacking clearly winning?

A: Because XGBoost was already the strongest base model, and the other two base models (random forest, sklearn gradient boosting) weren't different enough from it, or accurate enough on their own, to pull the combined prediction meaningfully past XGBoost's ceiling on this dataset.

Q: What's the practical tradeoff of deploying a voting or stacking ensemble in production versus a single model?

A: Ensembles require training, storing, and serving multiple models at inference time, increasing latency, memory footprint, and maintenance complexity. That cost has to be justified by a real accuracy or robustness gain over the best single model.

Interview Preparation — Theoretical Questions

Q: Why does ensembling reduce error most effectively when base models' errors are uncorrelated?

A: If two models make errors on the exact same rows in the exact same direction, combining them reproduces the same error. If their errors are independent or negatively correlated, a majority vote or averaged probability is more likely to be correct on any given row, since it takes agreement from multiple independent sources of evidence rather than trusting one potentially wrong model.

Q: Conceptually, how is stacking different from simply averaging (soft voting) in terms of what it can learn?

A: Soft voting applies a fixed, equal-weighted averaging rule to every row regardless of context. Stacking's meta-learner can learn conditional, non-uniform combination rules — for example, trusting model A more when model B and C disagree, or weighting XGBoost's prediction more heavily than the random forest's overall — because it's an actual trained model rather than a fixed arithmetic rule.

Q: Why would training a meta-learner directly on in-sample base-model predictions (without cross-validation) produce an overly optimistic ensemble?

A: In-sample predictions are made by base models that have already memorized (to some degree) the correct answers for those exact rows during their own training. A meta-learner trained on such predictions would essentially learn to trust suspiciously accurate signals that won't be available at real prediction time on genuinely unseen data, causing the ensemble to overfit and underperform in production relative to its apparent training-time accuracy.

Q: Why might a simple hard-voting ensemble sometimes outperform a more sophisticated stacking ensemble on a small dataset?

A: Stacking's meta-learner is itself a model that must be trained and can overfit, especially with limited data to estimate out-of-fold predictions reliably across folds. A simpler, parameter-free combination rule like majority voting has no additional parameters to overfit, making it more robust when the dataset is too small to reliably support a second layer of learning.

Q: How does the bias-variance tradeoff apply differently to voting versus stacking?

A: Voting is a fixed averaging or majority rule with no learnable parameters, so it primarily reduces variance by smoothing over individual base models' idiosyncratic errors without adding model complexity. Stacking introduces a trainable meta-learner, which can further reduce bias by learning non-uniform combination weights, but at the cost of adding its own variance risk if the meta-learner overfits the out-of-fold predictions.

Q: Why is it misleading to judge an ensemble purely by whether it beats the single best base model on one test set?

A: A single test set gives one noisy estimate of generalization performance; an ensemble that ties or slightly underperforms the best base model on one split might still generalize more robustly across many different random splits or shifted data distributions, because it isn't relying on any single model's specific failure modes. Conclusions about ensembling should ideally be checked across multiple splits or cross-validation folds, not one train/test call.

Q: What does it mean for a meta-learner to 'learn which base model to trust more,' and why can't a hard-voting scheme do the same thing?

A: The meta-learner assigns learned, data-driven weights (via its own coefficients, in the logistic regression case) to each base model's prediction, effectively learning that one base model's output is more reliably correlated with the true label than another's, possibly even conditionally on the other models' outputs. Hard voting has no such mechanism — it treats every base model's vote as exactly equal regardless of how historically trustworthy that model's predictions have been.

Q: Why does adding more, weaker base models to an ensemble not always improve it, even though ensembling theory favors combining more independent estimators?

A: The benefit of combining independent estimators assumes each addition contributes genuinely independent, unbiased error. A weak or highly correlated additional base model can dilute the stronger models' influence in a voting scheme, or add noisy input features to a meta-learner in a stacking scheme, potentially making the combined prediction worse rather than better — quality and diversity of base models matter more than raw quantity.